Infrastructure as Code

Integration, Management & Cloud-native Patterns

Ron C. Chiang

University of St. Thomas



Agenda

- Cloud-native application architecture patterns
- AWS
  - Cloud Watch
  - EC2 Auto-scaling
  - SNS
  - SQS
  - SES
  - Route 53
  - Elastic Beanstalk

Cloud-native application architecture

- The way we build applications for the cloud is fundamentally different than the way we traditionally built apps.

- Many legacy applications were never designed to run in cloud platforms, and therefore don't run very well.

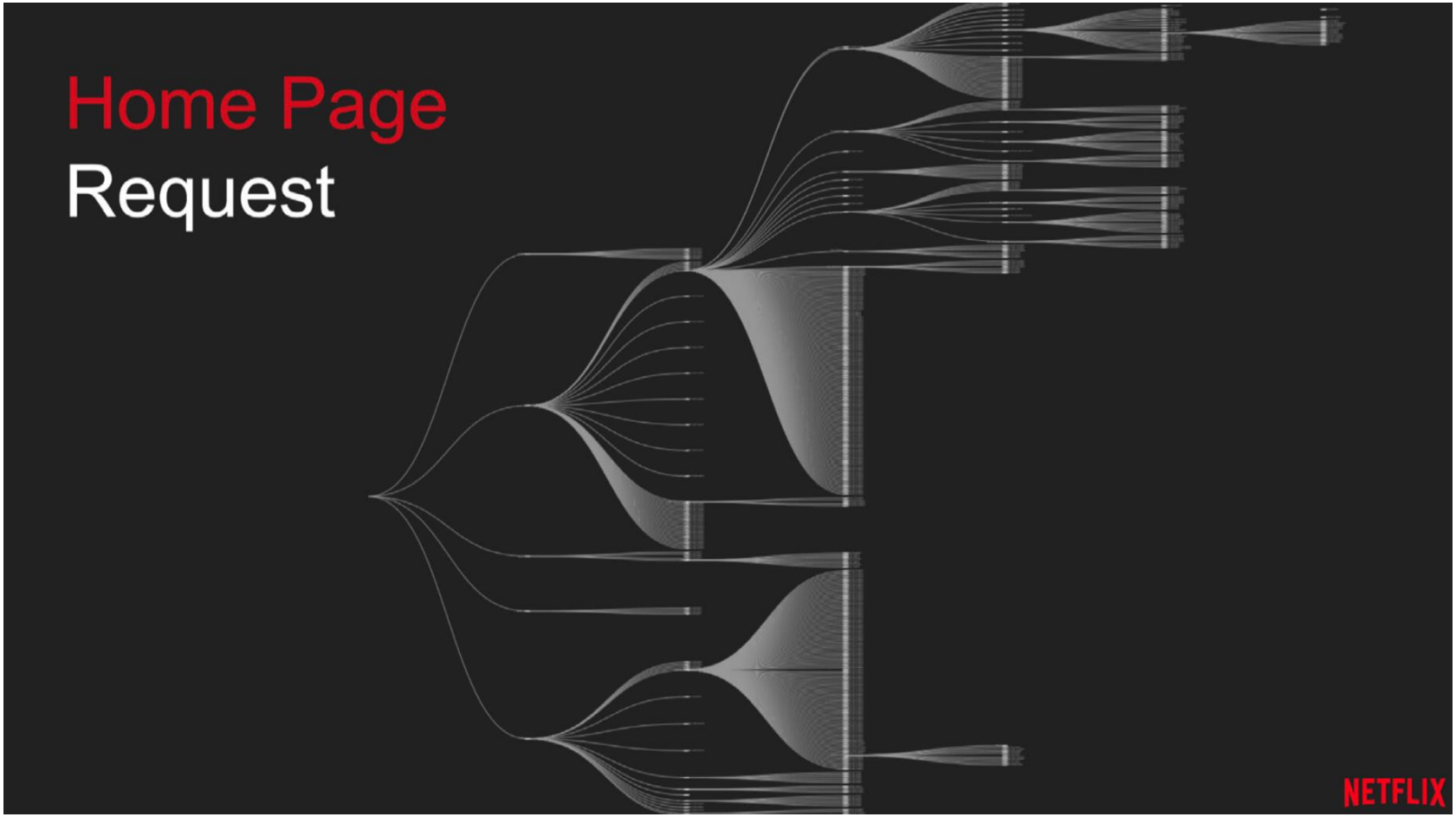
- Some legacy applications can move to the cloud, but cloud applications cannot move to traditional datacenters. *

* AWS Outpost allows you to stand up a rack of servers in your private datacenter to run a limited set of AWS services.

Cloud-native application architecture

- Cloud-native application characteristics:
  - Composed of multiple small services (micro-services) instead of a single monolithic application.
  - Highly dependent on network layer for application service communications.
  - Leverage managed services supported by the cloud provider.
  - Deployed across multiple availability zones and regions for higher resiliency.

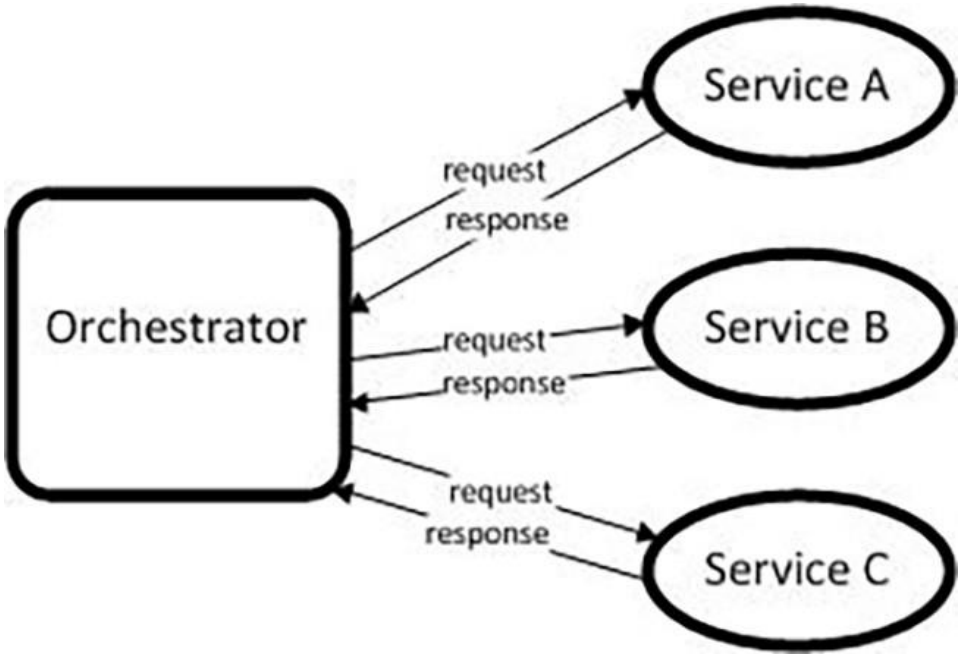
Cloud-native application architecture



Cloud-native application architecture

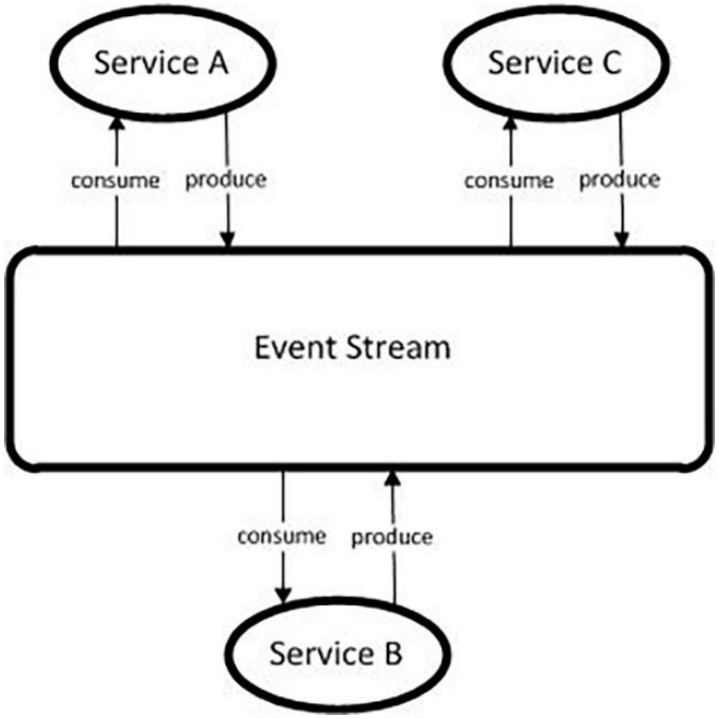
- The danger of cascading service calls:
  - Even if every service were available 99.999% of the time, with hundreds of services only 99% of the requests would be fulfilled.
- We can mitigate some of these issues using:
  - Retries (be careful!)
  - Caching
- Many modern applications are moving towards an event-driven model.

Orchestrator Pattern

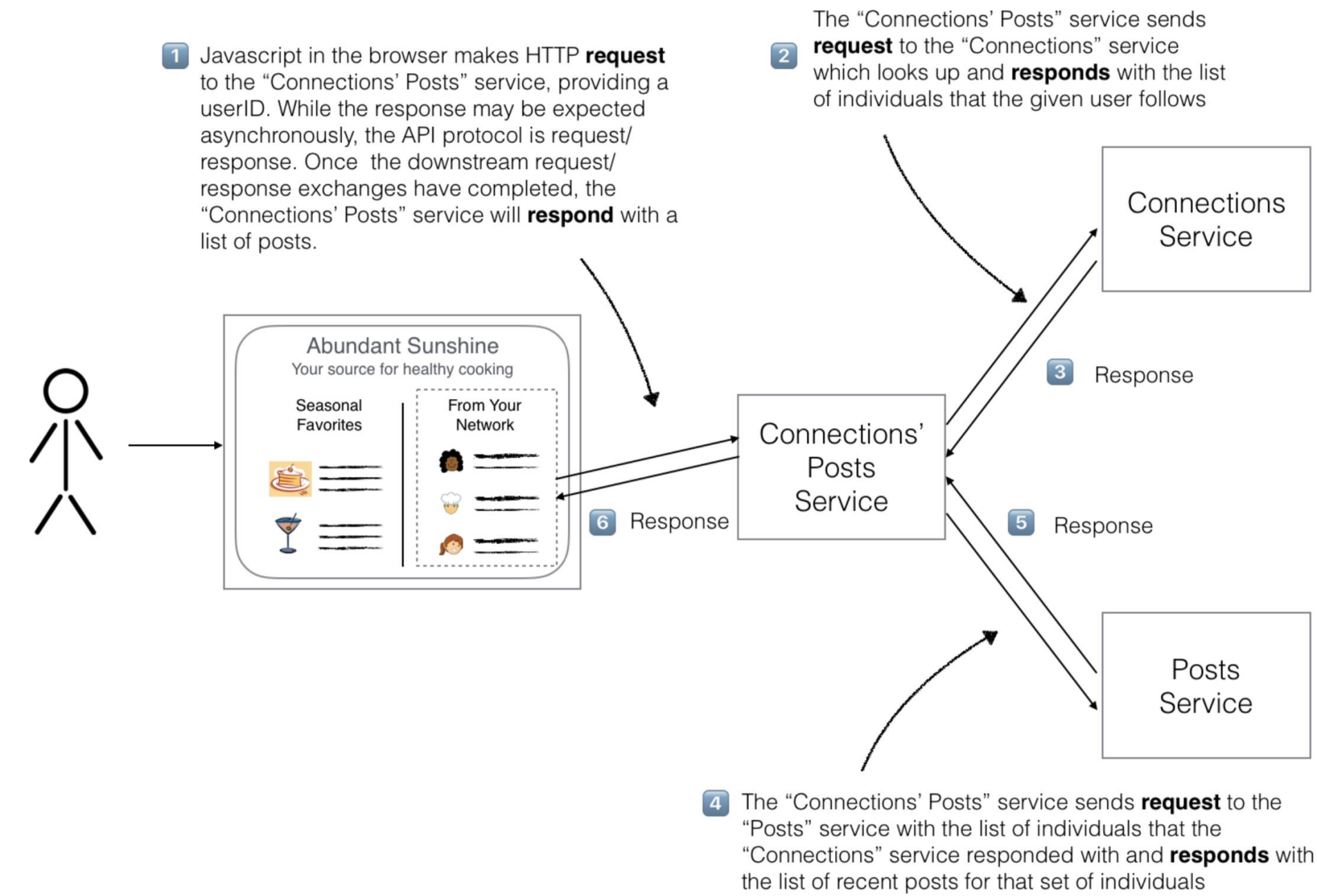


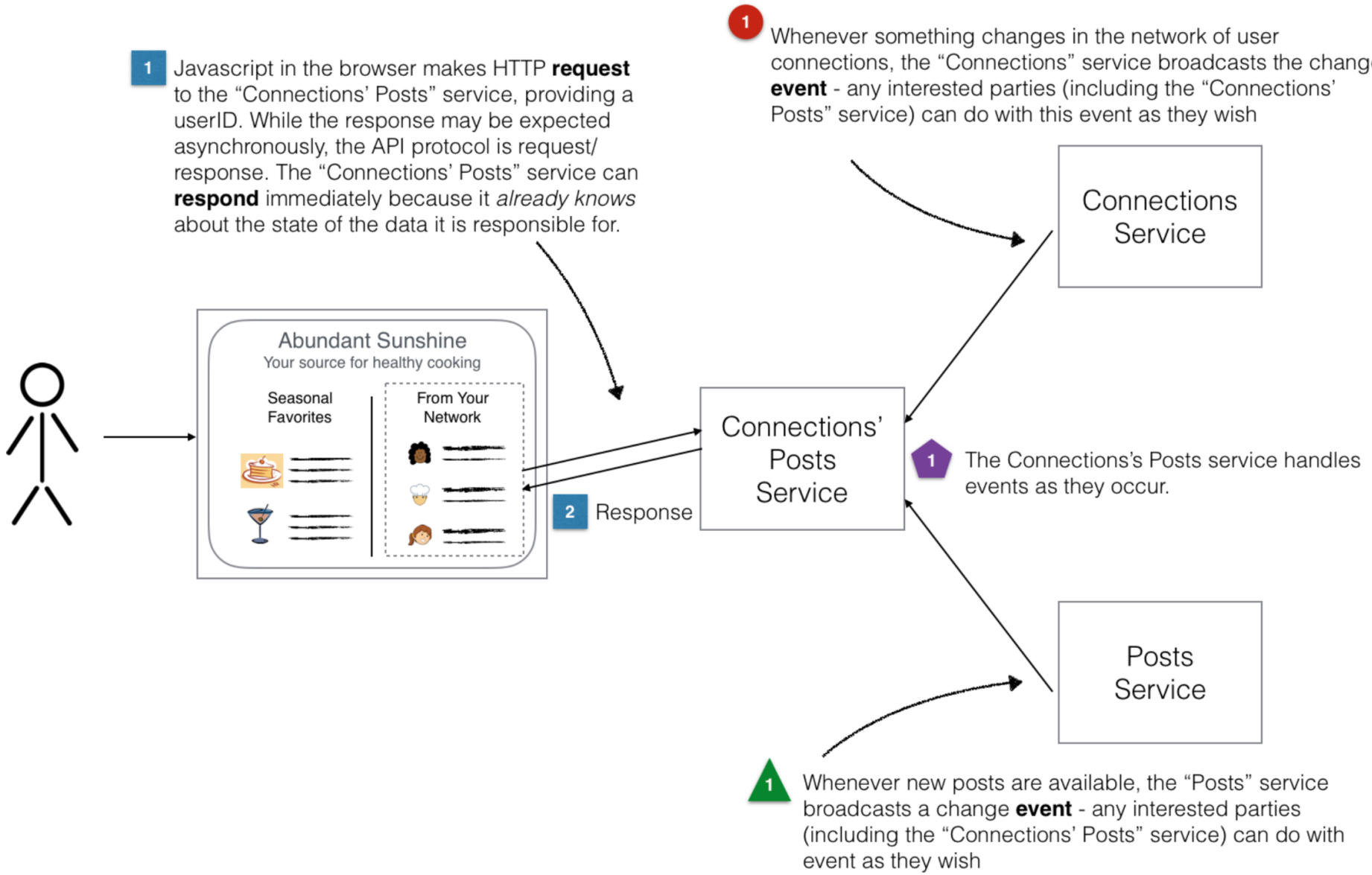
- Traditional request/response pattern
- Couples services together and creates dependencies.
- Single point of failure if orchestrating service goes down.

Reactive Pattern



- Event-driven communication allowing services to execute in parallel.
- No service dependencies making it easier to add/modify services.
- Many developers don't have experience with this architectural pattern.


1 Javascript in the browser makes HTTP **request** to the "Connections' Posts" service, providing a userID. While the response may be expected asynchronously, the API protocol is request/ response. Once the downstream request/ response exchanges have completed, the "Connections' Posts" service will **respond** with a list of posts.
2 The "Connections' Posts" service sends **request** to the "Connections" service which looks up and **responds** with the list of individuals that the given user follows
Abundant Sunshine
Your source for healthy cooking
Seasonal Favorites
From Your Network
Connections' Posts Service
Connections Service
3 Response
5 Response
6 Response
Posts Service
4 The "Connections' Posts" service sends **request** to the "Posts" service with the list of individuals that the "Connections" service responded with and **responds** with the list of recent posts for that set of individuals

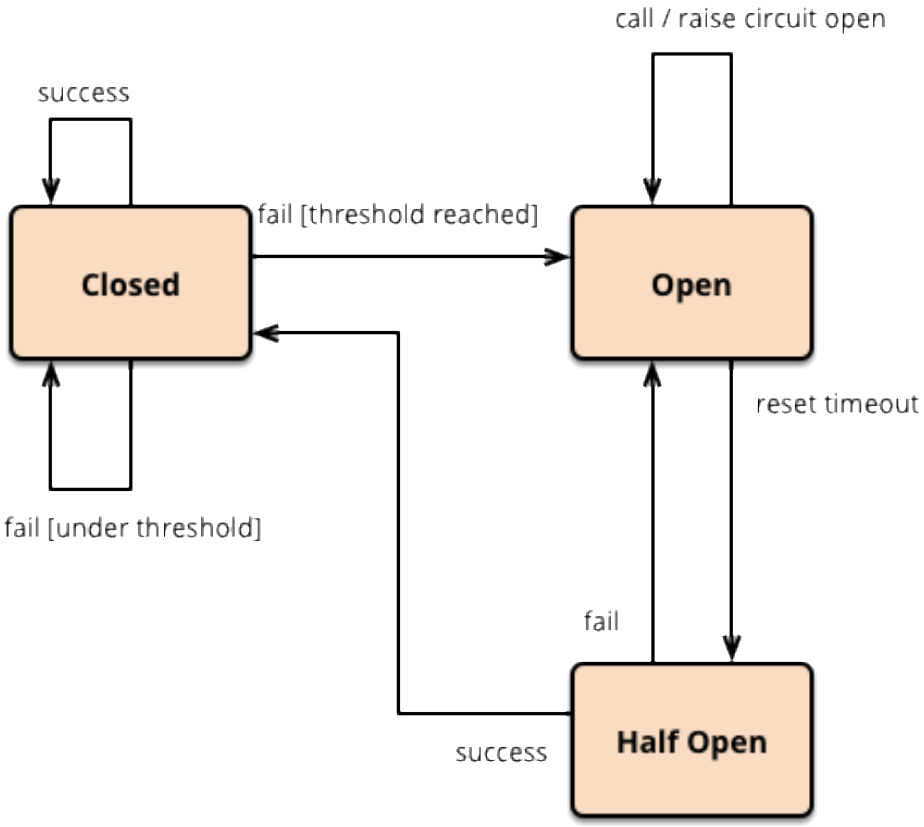

1
Javascript in the browser makes HTTP **request** to the "Connections' Posts" service, providing a userID. While the response may be expected asynchronously, the API protocol is request/response. The "Connections' Posts" service can **respond** immediately because it *already knows* about the state of the data it is responsible for.
1
Whenever something changes in the network of user connections, the "Connections" service broadcasts the change **event** - any interested parties (including the "Connections' Posts" service) can do with this event as they wish
Connections Service
Abundant Sunshine
Your source for healthy cooking
Seasonal Favorites
From Your Network
2
Response
Connections' Posts Service
1
The Connections's Posts service handles events as they occur.
Posts Service
1
Whenever new posts are available, the "Posts" service broadcasts a change **event** - any interested parties (including the "Connections' Posts" service) can do with event as they wish

Circuit Breaker pattern

- Cloud-native applications consist of services which call services which call services…
- Our platform may experience cascading service failures:
  - One or more services generating huge demand for a shared service.
  - A shared service which becomes unresponsive creates failures in other parts of the platform.

Circuit Breaker pattern

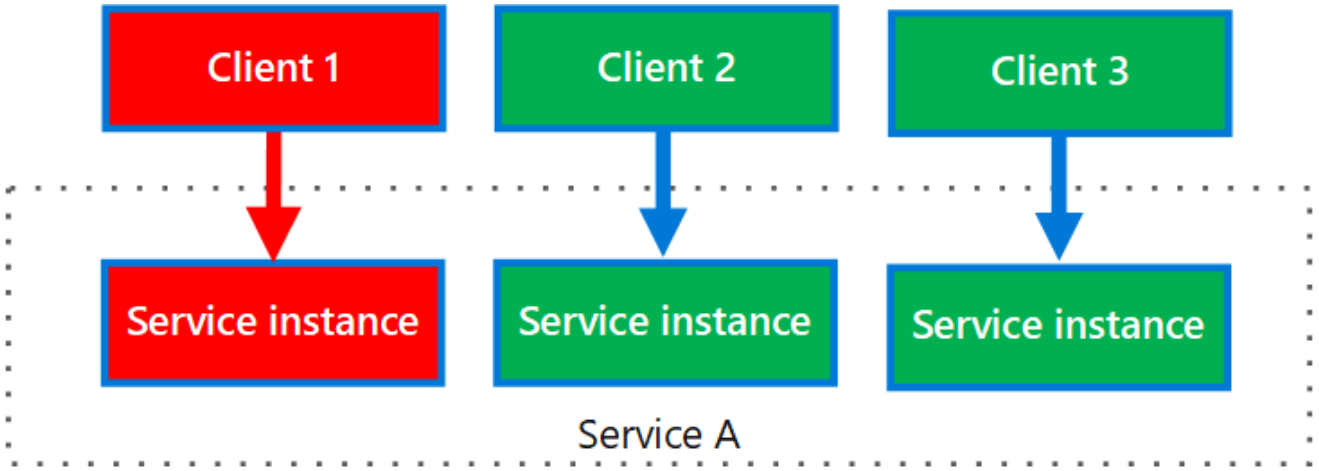
- Solution: services invoke one another through an abstraction layer which acts like an electrical circuit breaker.
- Service blocks requests when external service is no longer available.



Bulkhead Pattern

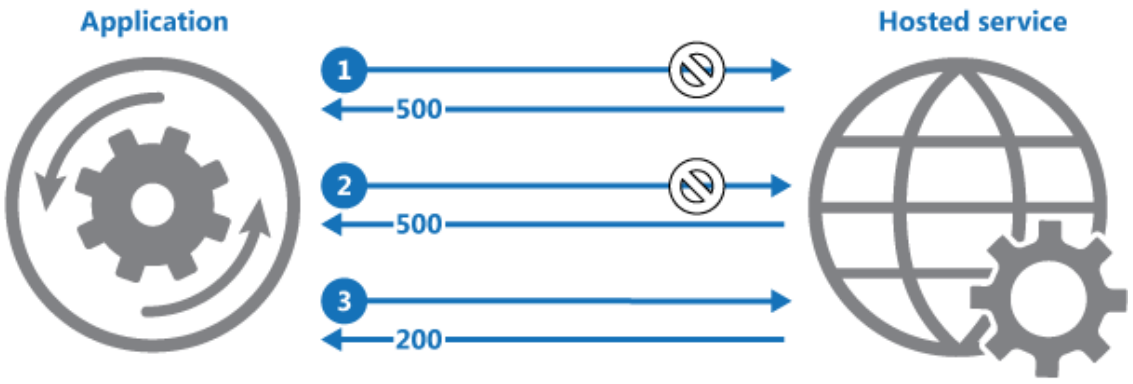
- In a shared platform one customer may generate demand which impacts other customers (noisy-neighbor problem).
  - Or maybe one service negatively impacts another service.

- Solution: partition resources into separate resource pools.



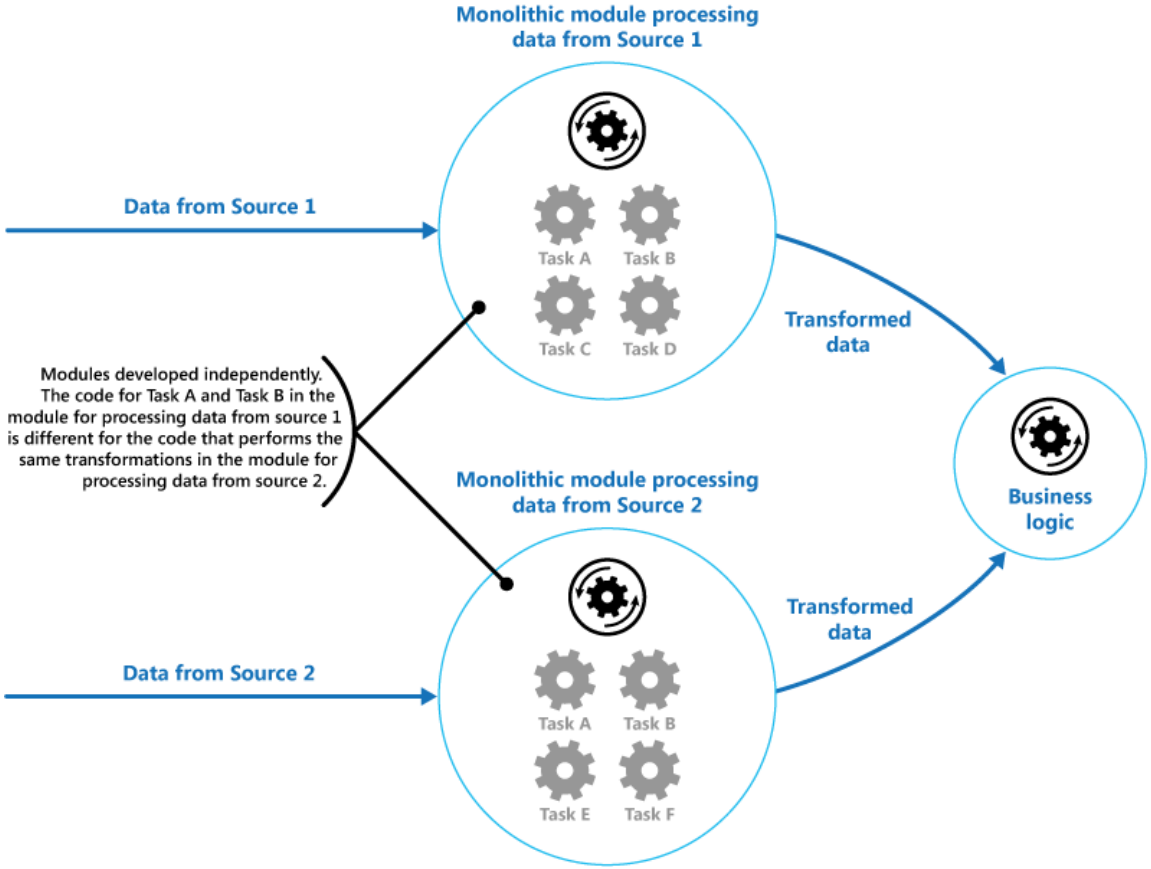
Retry Pattern

- Transient failures are common in cloud-native applications and should be handled elegantly.
- Solution: a service automatically tries to reconnect to another service after a connection failure.
  - We usually introduce a delay between re-connection attempts.
  - Many services trying to reconnect to a shared service may generate a retry storm!
  - In practice we always try to implement an exponential back-off delay when retrying connections.



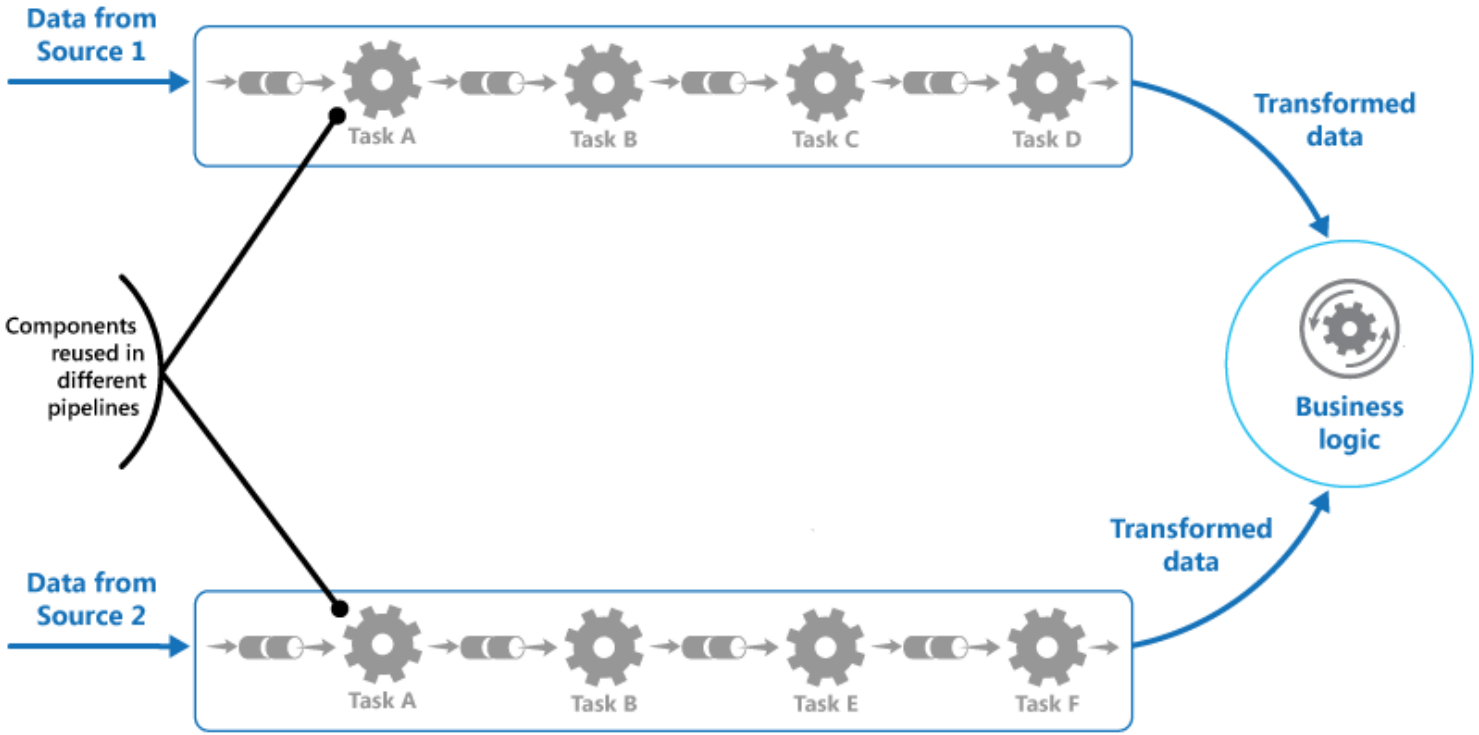
Pipes and Filters Pattern

- A monolithic application may perform a series of processing steps on a data set.



Pipes and Filters Pattern

- Cloud-native apps partition processing into separate components (called filters) that may be connected together to form a pipeline.



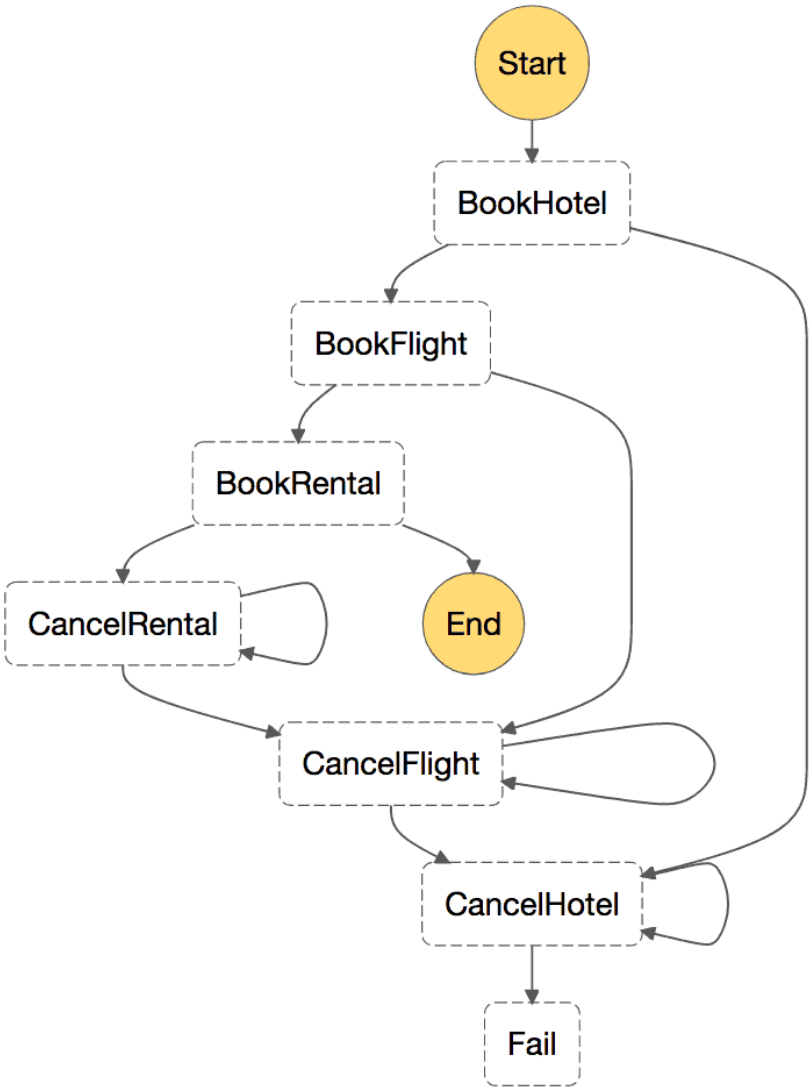
Compensating transaction (Saga) Pattern

- A transaction may involve multiple distributed services.

```
Begin transaction
    Start book hotel request
    End book hotel request
    Start book flight request
    End book flight request
    Start book car rental request
    End book car rental request
End transaction
```

- What happens if a service fails in the middle of the transaction?
  - We can retry the transaction which means the actions need to be idempotent.
  - We can also try to roll back the transaction using a state machine and compensating actions (CancelHotel, CancelFlight, CancelRental)

Compensating transaction (Saga) Pattern



CloudWatch

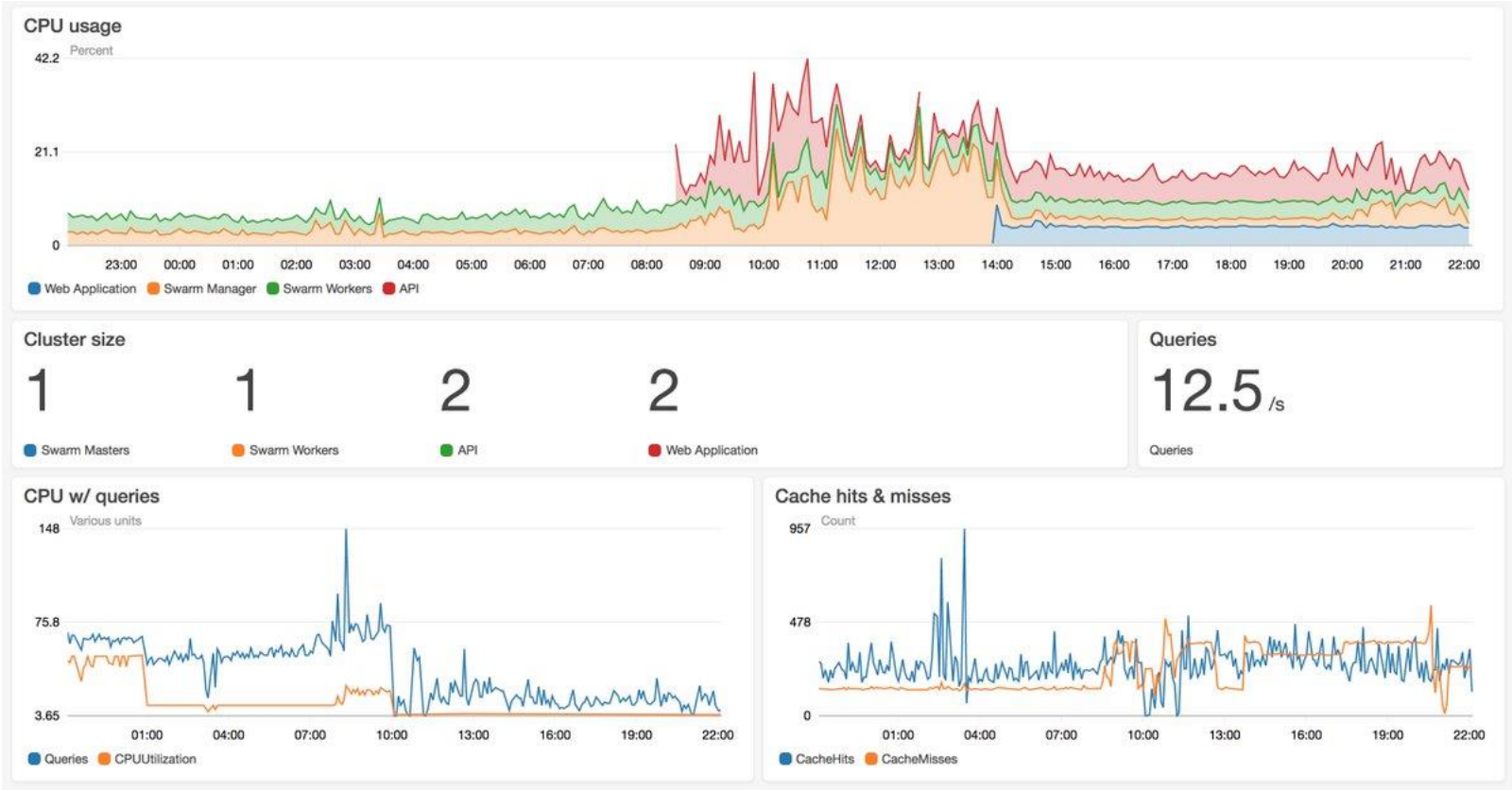


- **CloudWatch** is a monitoring and messaging service for AWS resources.
  - Track performance metrics
  - Collect log messages
  - Set alarms based on predefined metric thresholds
  - Generate event messages

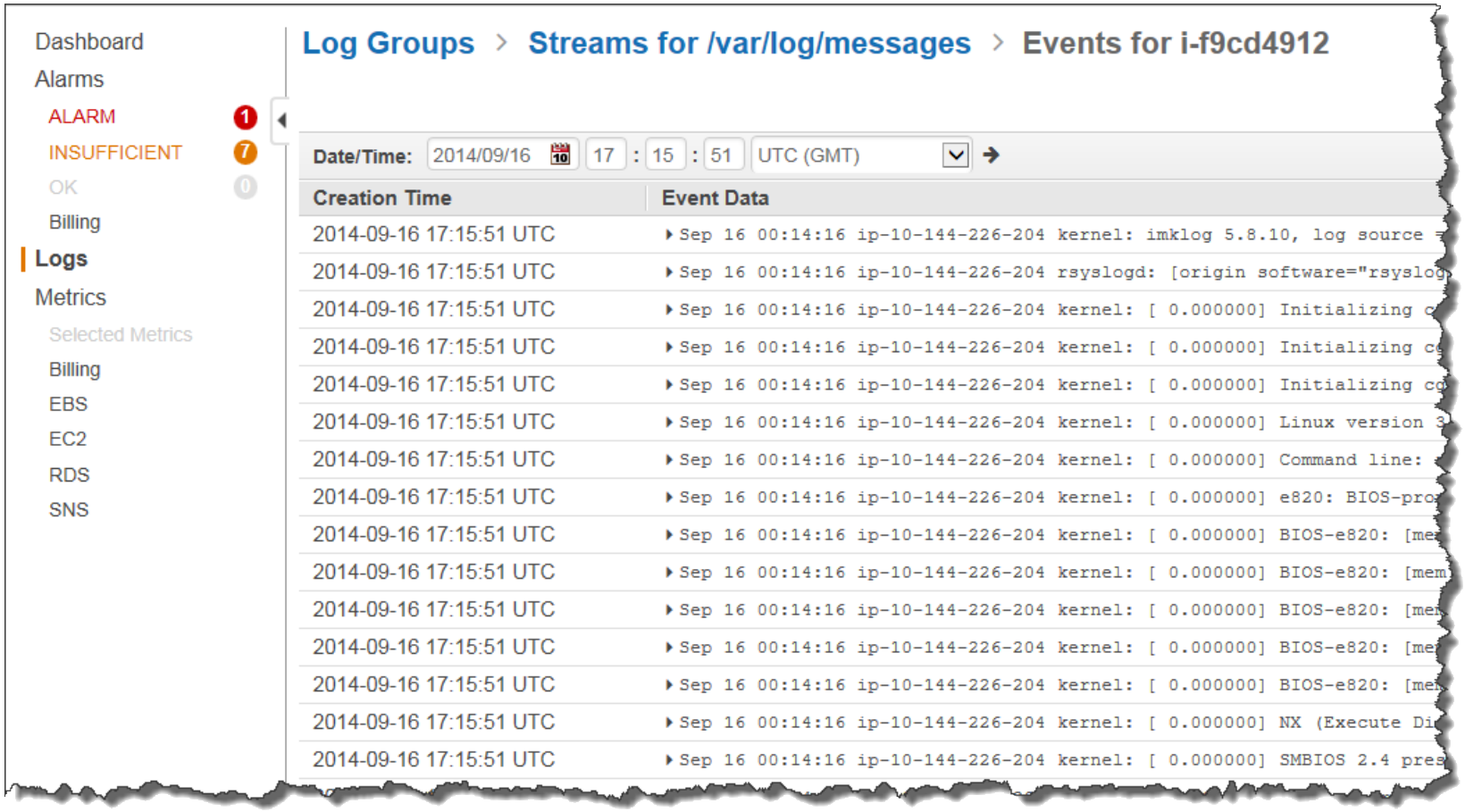
- Metric data is stored for up to 15 months with gradually decreasing granularity.

- Basic monitoring is free, detailed monitoring costs extra.
  - Increased interval from 5 min to 1 min
  - Monitor custom app metrics using an API

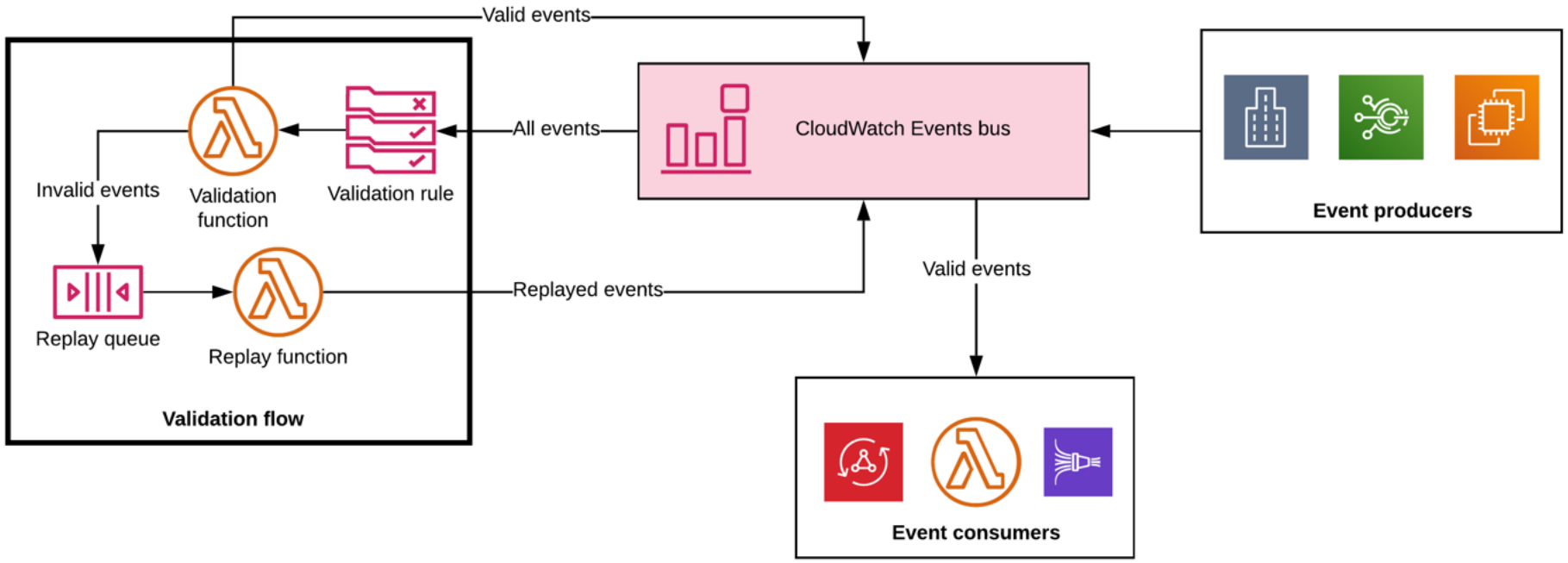
CloudWatch Metrics



CloudWatch Logs



CloudWatch Events



CloudTrail

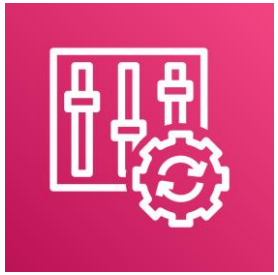


- A service that logs all AWS API calls for tracking, reporting and compliance purposes.

- A stream of logging data is called a **trail**.
  - Log data is stored in a designated S3 bucket.
  - Trail can contain logs from all regions or a specified region.
  - Maximum of 5 trails per region.

- Ability to setup notification when specific API calls are made.
  - Example: Notify when API call adds or removes EC2 instances.

AWS Config



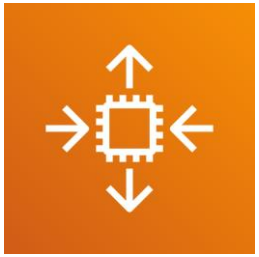
- A service that evaluates and documents the configuration of AWS resources.
  - Can review resources globally or just in a specific region.
  - Can review all resources or just selected ones.

- Use cases:
  - Take snapshots of resource configurations to review changes over time.
  - Audit resource configuration for compliance purposes.
    - Verify that all EBS volumes are encrypted.
    - Verify that resources are tagged properly.

- Reports are stored in a designated S3 bucket.

- Service can send notifications if a resource configuration changes.

EC2 Auto Scaling



- **Auto scaling** allows you to scale EC2 capacity up or down based on defined conditions.
  - Ensure a certain number of instances are always running.
  - Increase or decrease number of instances to support current demand and save costs.

- Only billed for instances created by Auto Scaling, not the service itself.

EC2 Auto Scaling

- Two components of auto scaling configuration:
  - **Launch Configuration**: specifies the type of EC2 instance created during the auto scaling process.
    - Very similar to launching an EC2 instance, in this case just the configuration is created.
  - **Auto Scaling Group**: defines a set of EC2 instances and the the conditions which determine the number of instances in the set.

EC2 Auto Scaling

- Auto Scaling Group configurations:
  - **Maintain minimum number of instances**: automatically launches new instances if a current instance becomes unhealthy.
  - **Manual scaling**: user manually changes auto scaling group configuration and AWS adjust instance count to match new configuration.
  - **Dynamic scaling**: instances launched or removed based on CloudWatch metrics and alarms.

Auto Scaling Hands On



- Create a launch template.
- Create an auto scaling group.
  - Group size = 2
  - Keep group at its initial size
- Watch AWS automatically launch EC2 instances.
- Manually fail one of the instances and watch automatic recovery.

Simple Notification Service (SNS)



- SNS makes it easy to publish messages from an application and deliver them to subscribers or other applications.

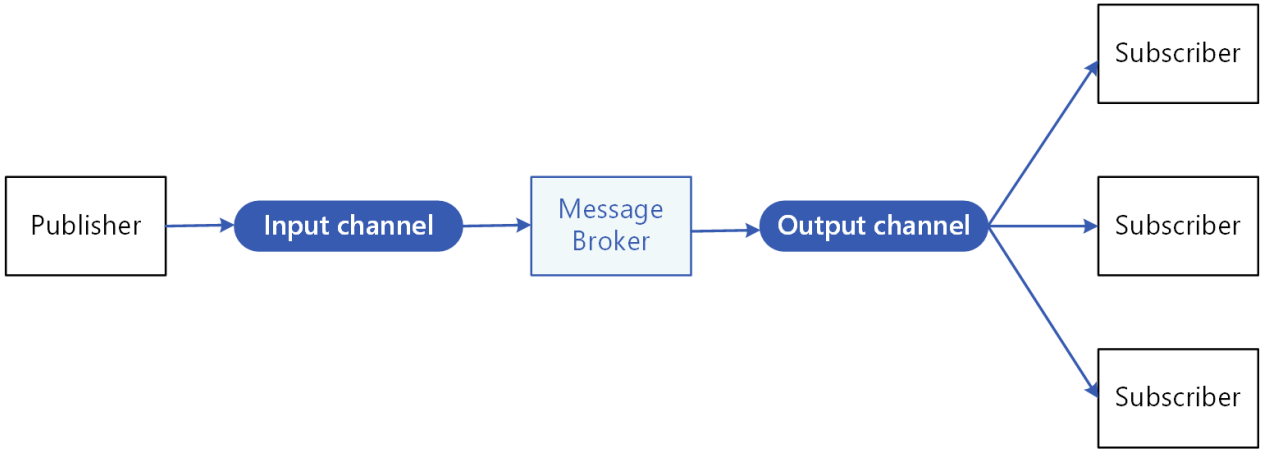
- Send messages via:
  - Email
  - SMS (text message)
  - Mobile push (Apple, Google, Fire OS, Windows)
  - SQS queues
  - HTTP endpoints

SNS

- A **topic** is created as an access point to allow multiple recipients to receive a message.
  - A single topic can support multiple endpoint types
- Messages published to SNS are stored in multiple AZs for redundancy.
- Recipients must subscribe to a topic in order to receive published messages.
- Messages are pushed to topic recipients.
- Use-cases: event notifications (triggers, alerts, auto scaling notifications, etc.)

SNS

- **Publisher-Subscriber** (pub/sub) pattern:
  - Allow application to provide events to multiple subscriber applications asynchronously.
  - The publisher and subscriber applications don't have to be aware of one another (loose coupling).



Simple Queue Service (SQS)

- SQS is a distributed queue system that allows messages generated by one system component to be consumed by another.
  - First web service launched by Amazon in 2005.

- Asynchronous application integration: message producers are decoupled from consumers



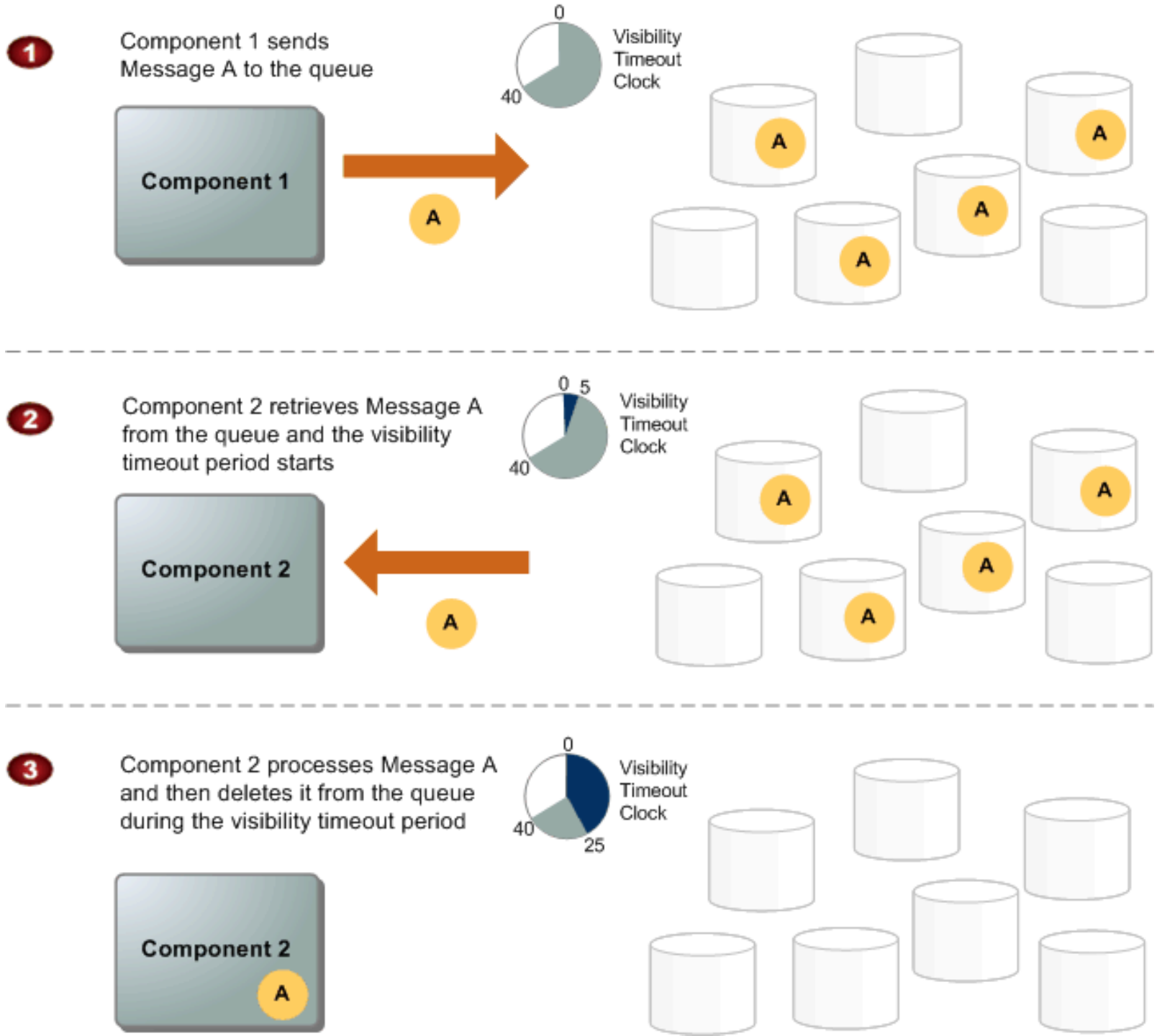
SQS

- SQS use cases:
  - Create thumbnails from user uploaded images or apply watermarks.
  - Support payment process workflow for orders.
  - Scale bursty applications that are trying to balance data consumption with backend data processing.

- Limits:
  - Messages up to 256KB in size
  - Messages can be stored in queue up to 14 days (default is 4 days)

SQS

- Two types of queues:
  - **Standard** queues (default)
    - Messages delivered at least once (but possibly more than once!).
    - Messages may occasionally be delivered out of order.
    - Action taken on message should be **idempotent** (repeat action multiple times with same result).
    - Nearly unlimited transactions per second.
  - **FIFO** (First-in-first-out)
    - Messages delivered exactly once.
    - Messages are delivered strictly in order.
    - Supports message groups to create separate groups of ordered messages within same queue.
    - Limited to 300 transactions per second.

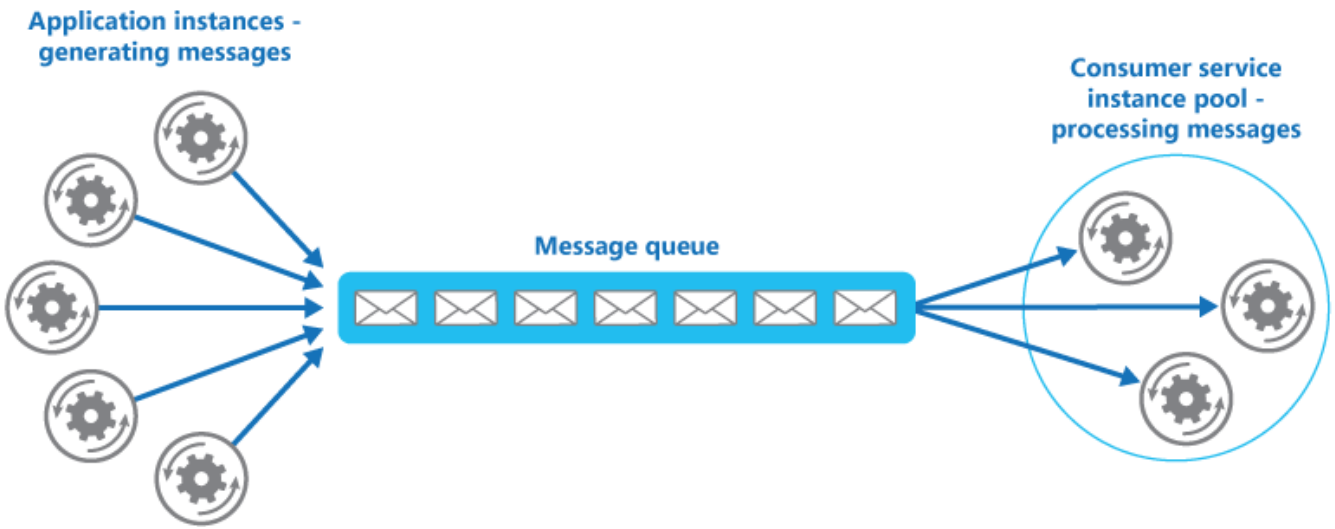

1
Component 1 sends Message A to the queue
Component 1
A
0
40
Visibility Timeout Clock
A
A
A
A
2
Component 2 retrieves Message A from the queue and the visibility timeout period starts
Component 2
A
0
5
40
Visibility Timeout Clock
A
A
A
A
3
Component 2 processes Message A and then deletes it from the queue during the visibility timeout period
Component 2
A
0
40
25
Visibility Timeout Clock

SQS

- Visibility timeout can be set up to 12 hours.
- Because messages are pulled, requests must be made every few seconds to see if anything in the queue.
  - 1 request every 5 sec = 12*60*24*7*4.3= 520,128/mo
  - 4 backend workers > 2 million requests/mo
  - Wasteful requests if queue is empty at times
- **Long polling**: keep message pull call open for up to 20 seconds, streaming in new messages as they arrive.

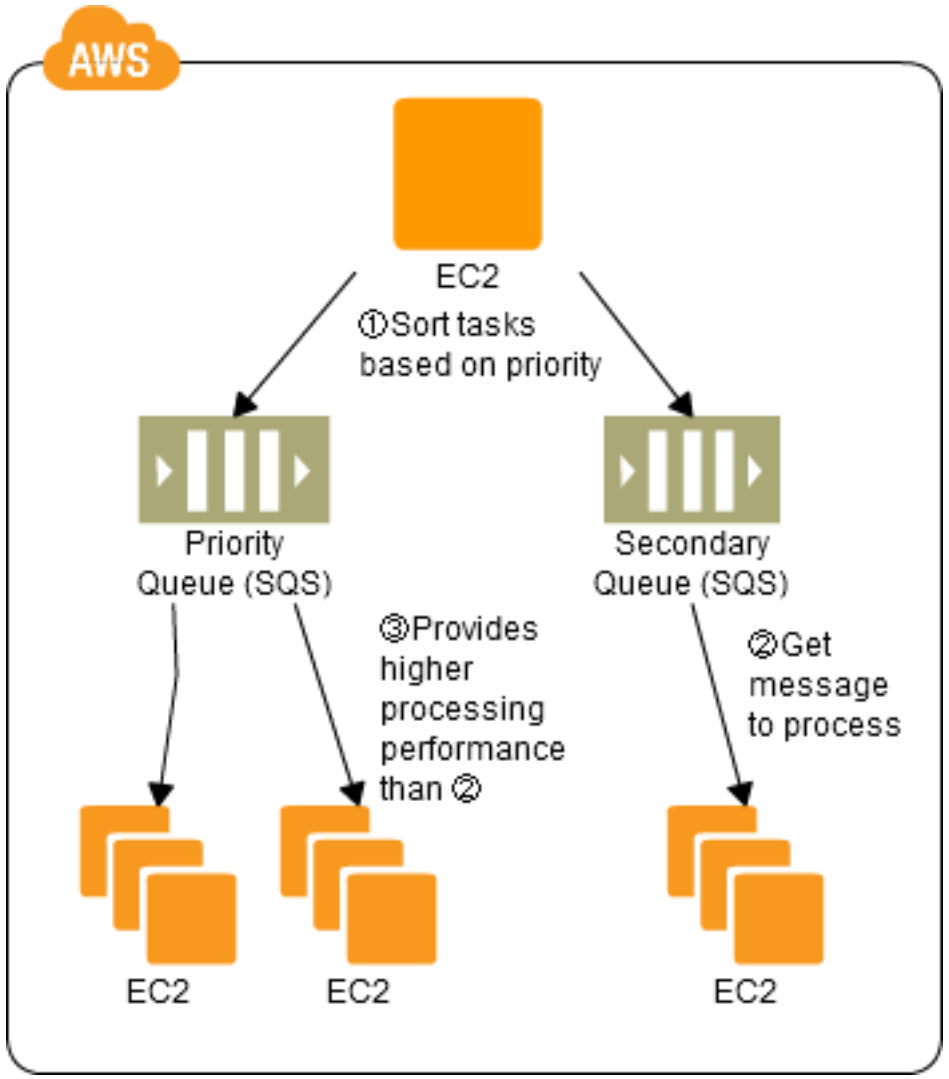
SQS

- **Competing Consumers** pattern:
  - Allow multiple applications to read from the same message queue.
  - Process requests independently in parallel.
  - Highly resilient, but no guarantee of order.

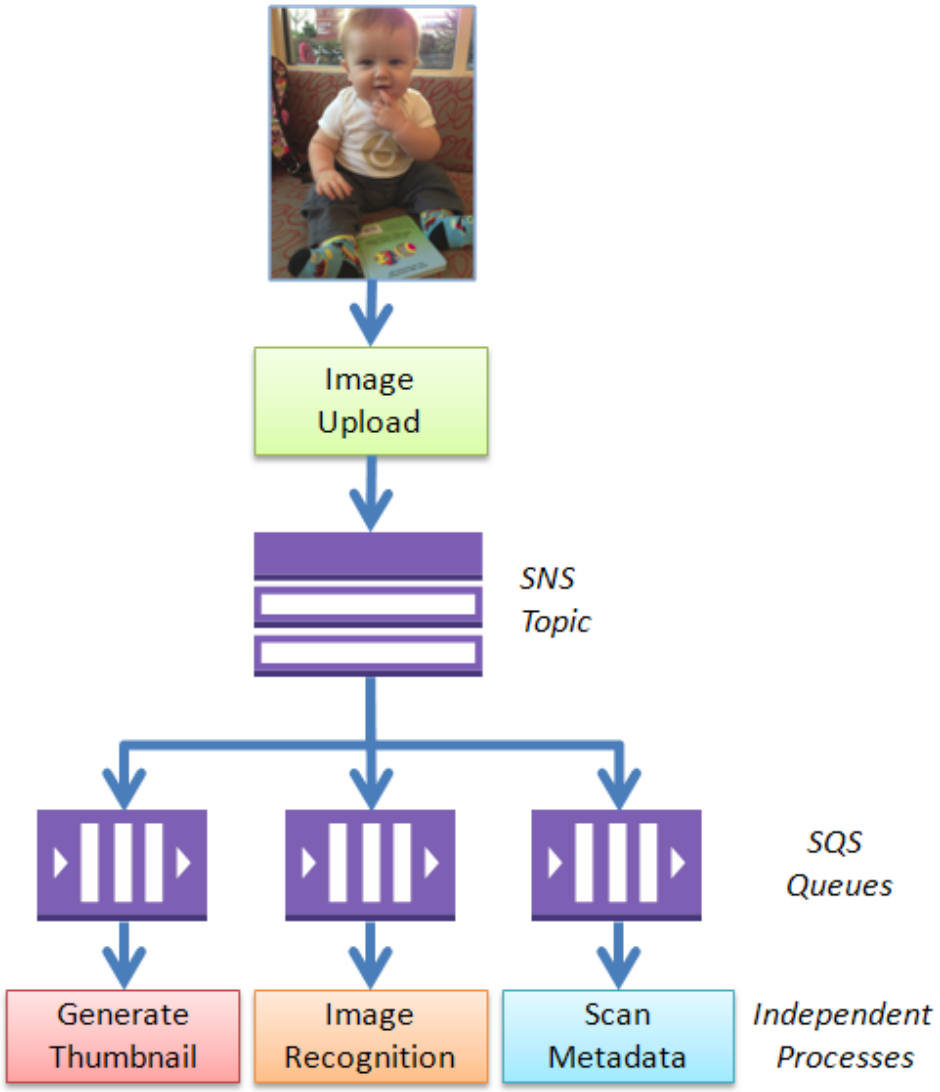


SQS Priority Queuing

SQS doesn't support the concept of prioritization, but you can use multiple queues to accomplish the same thing.



SQS Fan-out


Image Upload
SNS Topic
SQS Queues
Generate Thumbnail
Image Recognition
Scan Metadata
Independent Processes

SQS Hands On



- Create an SQS queue.
- Add a message to the queue with a publisher app.
- Check for messages on the queue.
- Pull a message from the queue with a subscriber app.

Simple Email Service (SES)



- SES is a scalable email service to support sending and receiving email from AWS.
- AWS customers must get approval to run mail servers (smtp) on EC2 instances.
- Messages sent via standard SMTP, received messages are stored in S3.
- Billing is based on number of messages sent and received per month (first 62k sent free).
- AWS places a limit on number of messages sent/day, but you can request an increase.

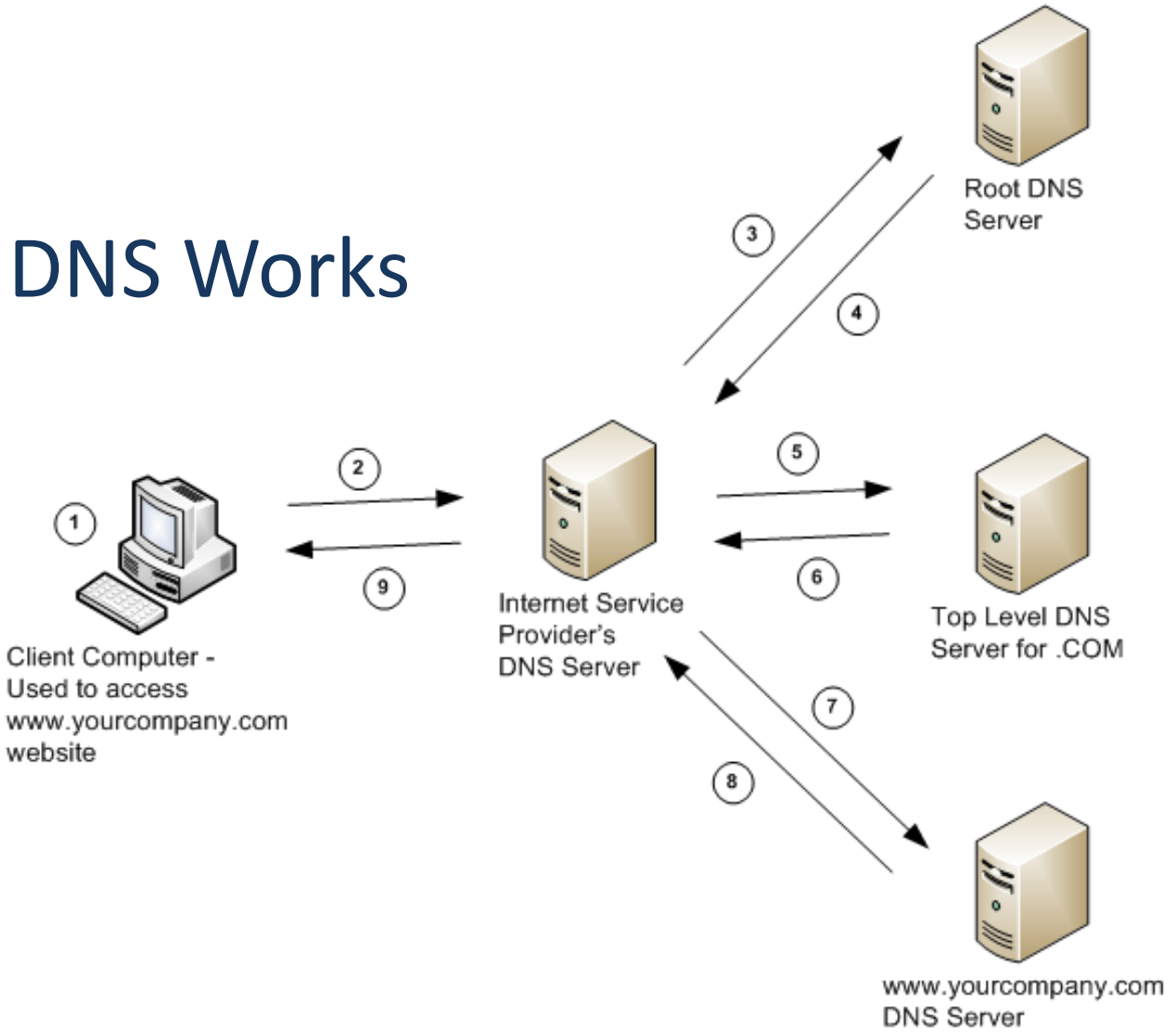
Route 53



- Scalable domain name hosting and registration service.
- Route 53 name is based on famous Route 66 highway across America, 53 is DNS port number.
- Domain Name Services (DNS) are a critical part of Internet infrastructure
  - DNS translates domain names (domain.com) into IP addresses.
  - Availability of DNS services directly impacts service availability.
    - Service isn't really "up" if nobody can find it.

Route 53

How DNS Works



Route 53

- AWS can serve as a domain registrar and name server.
- A **name server** provides all authoritative records for a domain name.
- Record types:
  - **Start of authority** (SOA): administrator name, version, and TTL information
  - **Name Server** (NS): used by top level servers to locate authoritative name server
  - **Address** (A): maps a hostname to an IP address
  - **CNAME**: maps a hostname to a fully qualified domain name
  - **Alias**: like a CNAME, but can use dynamic AWS targets
    - Example: Elastic load balancers use a dynamic endpoint, not an IP address

Route 53 Routing Policies

- Routing policies are used to distribute traffic over multiple regions using dynamic DNS responses.
- **Simple**: default policy which maps a DNS request to one or more addresses (random).
- **Multivalue Answer**: same as simple policy except Route 53 will perform health check and only respond with healthy records.
- **Weighted**: distribute traffic over multiple regions based on the weighting assigned to each region.
  - Example: 85% us-east, 15% us-west)
  - May be useful for A/B testing
- **Latency**: distribute traffic by directing user to the lowest latency region.
  - Example: US customers directed to us-east and European customers direct to eu-west

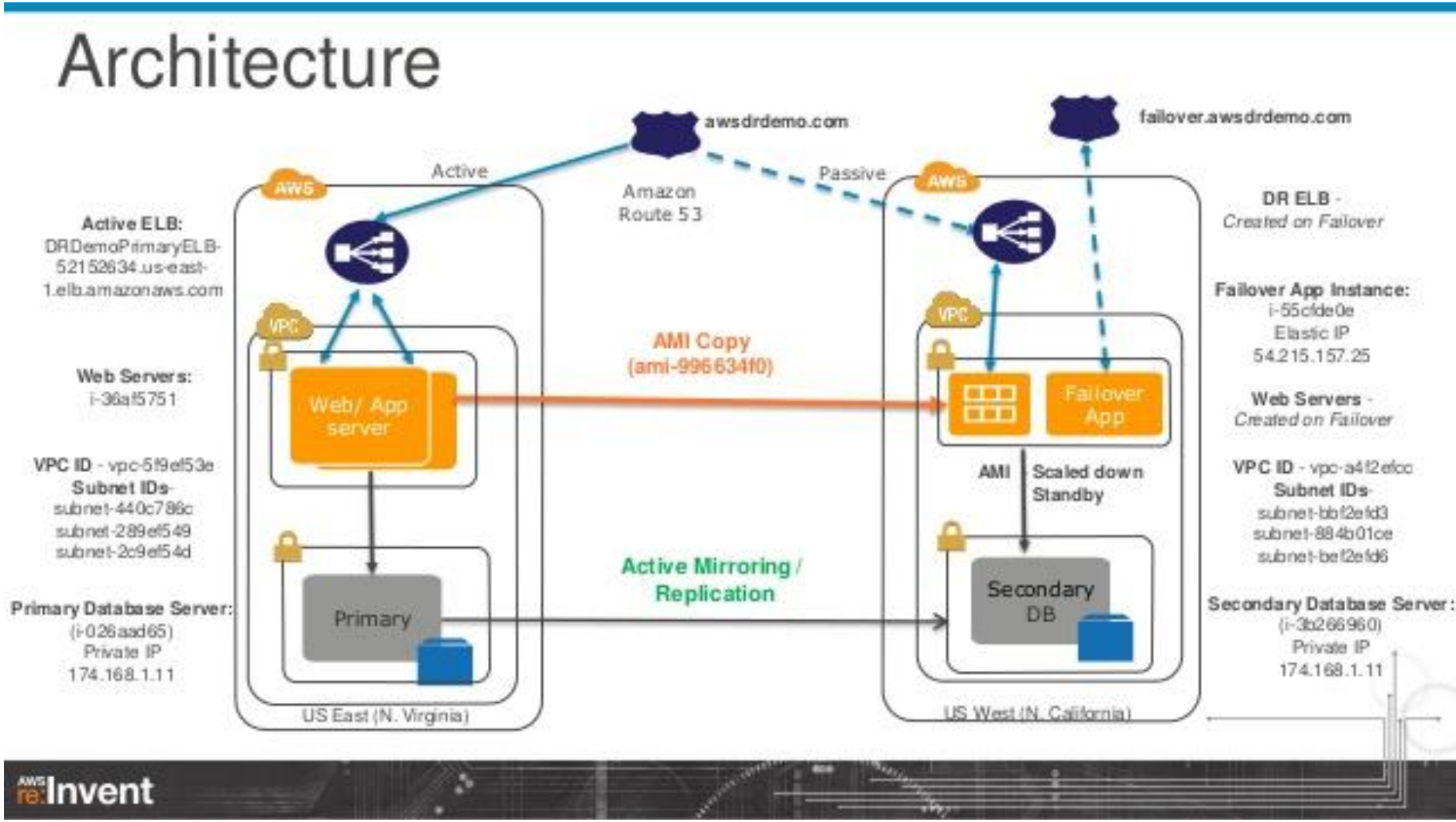
Route 53 Routing Policies

- **Failover**: monitor health of services and fail over to a secondary region if health check fails.
  - Example: us-east is primary, us-west is secondary
  - Primarily used for disaster recovery purposes, protect against failure of an entire region

- **Geolocation**: distribute traffic to a specific region based on physical location of user.
  - Example: All queries from European users go to eu-west
  - Primarily used to meet regulatory and data privacy requirements

- **Geoproximity Routing (Traffic flow):** route traffic based on both physical location of user and location of resources.
  - Able to define a bias setting which expands or shrinks geographic region
  - Must use *Route 53 traffic flow* when enabling this configuration

DR Architecture Using Route 53



Elastic Beanstalk



- PaaS which automates the configuration and deployment of application stacks.
  - Automatically provisions and scales EC2, load balancing, networking, storage, and RDS.
  - Installs and manages application servers and frameworks.
  - AWS user just focuses on loading custom application.
- Benefit: designed primarily for application developers that don’t want to deal with infrastructure.

Elastic Beanstalk

- Application support:
  - Java
  - PHP
  - Python
  - Ruby
  - IIS/.NET
  - Node.js
  - Docker

- EB controls capacity of resources, but you can still access infrastructure.

Elastic Beanstalk Deployment Policies

- Deployment policies control how EB updates deployed application:
  - **All at once**: deploy application to all instances simultaneously (short service disruption).
  - **Rolling**: deploy new app in batches, reducing service capacity but causing no disruption.
  - **Rolling with additional batch**: launch new resources before batch updates so that service capacity isn't reduced.
  - **Immutable**: deploy new application to a fresh group of instances (blue-green deployment).